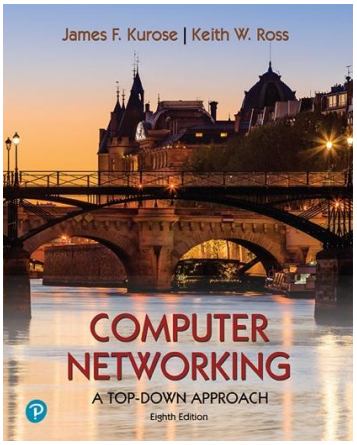
National University

Of Computer & Emerging Sciences Peshawar Campus

Wireshark Lab: 07
HTTP v8.1

Supplement to *Computer Networking: A Top-Down Approach, 8th ed.*, J.F. Kurose and K.W. Ross



"Tell me and I forget. Show me and I remember. Involve me and I understand." Chinese proverb

© 2005-2021, J.F Kurose and K.W. Ross, All Rights Reserved

In this lab, we'll explore several aspects of the HTTP protocol: the basic GET/response interaction, HTTP message formats, retrieving large HTML files, retrieving HTML files with embedded objects, and HTTP authentication and security. Before beginning these labs, you might want to review Section 2.2 of the text.[1]

1. The HTTP CONDITIONAL GET/response interaction

Recall from Section 2.2.5 of the text, that most web browsers perform object caching and thus often perform a conditional GET when retrieving an HTTP object. Before performing the steps below, make sure your browser's cache is empty[2]. Now do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html
  Your browser should display a very simple five-line HTML file.
- Quickly enter the same URL into your browser again (or simply select the refresh button on your browser)
- Stop Wireshark packet capture, and enter "http" (again, in lower case without the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

---

[1] References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach, 8h ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020.* Our authors' website for this book is http://gaia.cs.umass.edu/kurose_ross You'll find lots of interesting open material there.

[2] See https://www.howtogeek.com/304218/how-to-clear-your-history-in-any-browser/ for instructions on clearing your browser cache.

If you're unable to run Wireshark on a live network connection (or unable to get your browser to issue an If-Modified-Since field on the second HTTP GET request), you can download a packet trace that was created when the steps above were followed.[3] Answer the following questions:

1. Inspect the contents of the first HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE" line in the HTTP GET? (0.3)

```
Hypertext Transfer Protocol
  GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n
      Request Method: GET
      Request URI: /wireshark-labs/HTTP-wireshark-file2.html
      Request Version: HTTP/1.1
    Host: gaia.cs.umass.edu\r\n
    Connection: keep-alive\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36 Edg/135.0.0.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: en-US,en;q=0.9\r\n
    \r\n
    [Response in frame: 399]
    [Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html]
```

No, the first GET does not include the 'If-Modified-Since' header.

2. Inspect the contents of the server response. Did the server explicitly return the contents of the file? How can you tell? (0.3)

```
Hypertext Transfer Protocol
  HTTP/1.1 200 OK\r\n
  Date: Thu, 17 Apr 2025 07:39:35 GMT\r\n
  Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl/v5.16.3\r\n
  Last-Modified: Thu, 17 Apr 2025 05:59:01 GMT\r\n
  ETag: "173-632f31935826a"\r\n
  Accept-Ranges: bytes\r\n
```

Yes. The server responded with '200 OK' and included the file contents.

3. Now inspect the contents of the second HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE:" line in the HTTP GET[4]? If so, what information follows the "IF-MODIFIED-SINCE:" header? (0.4)

---

[3] If you're unable to run Wireshark on a live network connection, you can download the zip file http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip and extract the trace file http-wireshark-trace2-1.

[4] *Hint:* ideally, you should see an If-Modified-Since header since you've just downloaded this page a few seconds ago. However, depending on the browser you're using, and the format of the server's earlier response to your initial GET, your browser may not include an If-Modified-Since even if the document has been downloaded and caches. The Chrome browser is pretty good at regularly using If-Modified-Since. But Safari and Firefox are much more finicky about when to use If-Modified-Since. Life isn't always as easy in practice as it is in theory!

```
▸ Ethernet II, Src: LCFCElectron_0c:58:4f (8c:16:45:0c:58:4f), Dst: Cisco_4c:11:58 (2c:d0:2d:4c:11:58)
▸ Internet Protocol Version 4, Src: 172.16.4.135, Dst: 128.119.245.12
▸ Transmission Control Protocol, Src Port: 51062, Dst Port: 80, Seq: 599, Ack: 241, Len: 598
▾ Hypertext Transfer Protocol
  ▾ GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n
      Request Method: GET
      Request URI: /wireshark-labs/HTTP-wireshark-file2.html
      Request Version: HTTP/1.1
    Host: gaia.cs.umass.edu\r\n
    Connection: keep-alive\r\n
    Cache-Control: max-age=0\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: en-US,en;q=0.9\r\n
    If-None-Match: "173-632f31935826a"\r\n
    If-Modified-Since: Thu, 17 Apr 2025 05:59:01 GMT\r\n
    \r\n
    [Response in frame: 13822]
    [Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html]

Hypertext Transfer Protocol: Protocol
```

Yes, it includes a timestamp from the original response.

4. What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file? Explain. (1)

```
▸ Transmission Control Protocol, Src Port: 80, Dst Port: 51003, Seq: 223, Ack:
▾ Hypertext Transfer Protocol
  ▾ HTTP/1.1 304 Not Modified\r\n
      Response Version: HTTP/1.1
      Status Code: 304
      [Status Code Description: Not Modified]
      Response Phrase: Not Modified
    Date: Thu, 17 Apr 2025 07:32:01 GMT\r\n
    Expires: Thu, 17 Apr 2025 08:22:01 GMT\r\n
    Age: 454\r\n
    Last-Modified: Thu, 03 Apr 2025 14:18:00 GMT\r\n
    Cache-Control: public, max-age=3000\r\n
    Vary: Accept-Encoding\r\n
    \r\n
    [Request in frame: 395]
```

The server replied with '304 Not Modified', so the file was not resent

2. Retrieving Long Documents

In our examples thus far, the documents retrieved have been simple and short HTML files. Let's next see what happens when we download a long HTML file. Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer

- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html
  Your browser should display the rather lengthy US Bill of Rights.
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.

In the packet-listing window, you should see your HTTP GET message, followed by a multiple-packet TCP response to your HTTP GET request. Make sure you your Wireshark display filter is cleared so that the multi-packet TCP response will be displayed in the packet listing.

This multiple-packet response deserves a bit of explanation. Recall from Section 2.2 (see Figure 2.9 in the text) that the HTTP response message consists of a status line, followed by header lines, followed by a blank line, followed by the entity body. In the case of our HTTP GET, the entity body in the response is the *entire* requested HTML file. In our case here, the HTML file is rather long, and at 4500 bytes is too large to fit in one TCP packet. The single HTTP response message is thus broken into several pieces by TCP, with each piece being contained within a separate TCP segment (see Figure 1.24 in the text). In recent versions of Wireshark, Wireshark indicates each TCP segment as a separate packet, and the fact that the single HTTP response was fragmented across multiple TCP packets is indicated by the "TCP segment of a reassembled PDU" in the Info column of the Wireshark display.

Answer the following questions[5]:

5. How many HTTP GET request messages did your browser send? Which packet number in the trace contains the GET message for the Bill or Rights? (1)

No.	Time	Source	Destination	Protocol	Lengt	Info
1190	20.063233	fe80::7ddd:f9ba:e12…	fe80::5edc:d429:191…	HTTP/X…	807	POST /6d3d5cd1-9613-49a0-8d03-51320fee6e0d/ HTTP/1.1
1331	22.379559	172.16.4.135	128.119.245.12	HTTP	540	GET /wireshark-labs/HTTP-wireshark-file3.html HTTP/1.1
1363	22.604602	128.119.245.12	172.16.4.135	HTTP	535	HTTP/1.1 200 OK (text/html)

One Request Packet No 1331 contain get request

6. Which packet number in the trace contains the status code and phrase associated with the response to the HTTP GET request? (1)

1358	22.603799	128.119.245.12	172.16.4.135	TCP	60	80 → 51126 [ACK] Seq=1 Ack=487 Win=237 Len=0
1359	22.604294	128.119.245.12	172.16.4.135	TCP	1514	80 → 51126 [ACK] Seq=1 Ack=487 Win=237 Len=1460 [TCP PDU reassembled in 1363]
1360	22.604294	128.119.245.12	172.16.4.135	TCP	1514	80 → 51126 [ACK] Seq=1461 Ack=487 Win=237 Len=1460 [TCP PDU reassembled in 1363]
1361	22.604391	172.16.4.135	128.119.245.12	TCP	54	51126 → 80 [ACK] Seq=487 Ack=2921 Win=255 Len=0
1362	22.604602	128.119.245.12	172.16.4.135	TCP	1514	80 → 51126 [ACK] Seq=2921 Ack=487 Win=237 Len=1460 [TCP PDU reassembled in 1363]
1363	22.604602	128.119.245.12	172.16.4.135	HTTP	535	HTTP/1.1 200 OK (text/html)
1364	22.604650	172.16.4.135	128.119.245.12	TCP	54	51126 → 80 [ACK] Seq=487 Ack=4862 Win=255 Len=0

Packet No 1363 contain the Status code OK

[5] If you're unable to run Wireshark on a live network connection, you can download the zip file http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip and extract the trace file http-wireshark-trace3-1.

7. What is the status code and phrase in the response? (1)

```
▸ [4 Reassembled TCP Segments (4861 bytes): #1359(1460), #13
▾ Hypertext Transfer Protocol
  ▾ HTTP/1.1 200 OK\r\n
      Response Version: HTTP/1.1
      Status Code: 200
      [Status Code Description: OK]
      Response Phrase: OK
```

'OK' is the response Phrase

8. How many data-containing TCP segments were needed to carry the single HTTP response and the text of the Bill of Rights? (1)

1358	22.603799	128.119.245.12	172.16.4.135	TCP	60	80 → 51126 [ACK] Seq=1 Ack=487 Win=237 Len=0
1359	22.604294	128.119.245.12	172.16.4.135	TCP	1514	80 → 51126 [ACK] Seq=1 Ack=487 Win=237 Len=1460 [TCP PDU reassembled in 1363]
1360	22.604294	128.119.245.12	172.16.4.135	TCP	1514	80 → 51126 [ACK] Seq=1461 Ack=487 Win=237 Len=1460 [TCP PDU reassembled in 1363]
1361	22.604391	172.16.4.135	128.119.245.12	TCP	54	51126 → 80 [ACK] Seq=487 Ack=2921 Win=255 Len=0
1362	22.604602	128.119.245.12	172.16.4.135	TCP	1514	80 → 51126 [ACK] Seq=2921 Ack=487 Win=237 Len=1460 [TCP PDU reassembled in 1363]

3 data-containing TCP segments were needed to carry the single HTTP response

3. HTML Documents with Embedded Objects

Now that we've seen how Wireshark displays the captured packet traffic for large HTML files, we can look at what happens when your browser downloads a file with embedded objects, i.e., a file that includes other objects (in the example below, image files) that are stored on another server(s).

Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html
  Your browser should display a short HTML file with two images. These two images are referenced in the base HTML file. That is, the images themselves are not contained in the HTML; instead the URLs for the images are contained in the downloaded HTML file. As discussed in the textbook, your browser will have to retrieve these logos from the indicated web sites. Our publisher's logo is retrieved from the gaia.cs.umass.edu web site. The image of our 8th edition cover (one of our favorite covers) is stored at a server in France.
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.

Answer the following questions[6]:

9. How many HTTP GET request messages did your browser send? To which Internet addresses were these GET requests sent? (1)

52745	1129.454784	172.16.4.135	128.119.245.12	HTTP	540	GET /wireshark-labs/HTTP-wireshark-file4.html HTTP/1.1
52770	1129.677612	128.119.245.12	172.16.4.135	HTTP	1355	HTTP/1.1 200 OK (text/html)
52775	1129.720962	172.16.4.135	128.119.245.12	HTTP	486	GET /pearson.png HTTP/1.1
52833	1129.967309	172.16.4.135	178.79.137.164	HTTP	453	GET /8E_cover_small.jpg HTTP/1.1
52843	1130.116956	178.79.137.164	172.16.4.135	HTTP	225	HTTP/1.1 301 Moved Permanently
53530	1133.449668	fe80::5edc:d429:191…	fe80::9d88:7743:881…	HTTP/X…	807	POST /6deeb5d5-4014-4df8-ba3d-af81e6e731dd/ HTTP/1.1
53535	1133.454969	fe80::9d88:7743:881…	fe80::5edc:d429:191…	HTTP/X…	979	HTTP/1.1 200

Three HTTP GET request messages was sent from the browser.

Request No	Internet Address
1	128.119.245.12
2	128.119.245.12
3	178.79.137.164

10. Can you tell whether your browser downloaded the two images serially, or whether they were downloaded from the two web sites in parallel? Explain. (1)

52775	1129.720962	172.16.4.135	128.119.245.12	HTTP	486	GET /pearson.png HTTP/1.1
52833	1129.967309	172.16.4.135	178.79.137.164	HTTP	453	GET /8E_cover_small.jpg HTTP/1.1

Sicne The time stemp are different So ,both GET request were sent serially

4 HTTP Authentication

Finally, let's try visiting a web site that is password-protected and examine the sequence of HTTP message exchanged for such a site. The URL http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html is password protected. The username is "wireshark-students" (without the quotes), and the password is "network" (again, without the quotes). So let's access this "secure" password-protected site. Do the following:

- Make sure your browser's cache is cleared, as discussed above, and close down your browser. Then, start up your browser
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html
  Type the requested user name and password into the pop up box.

[6] If you're unable to run Wireshark on a live network connection, you can download the zip file http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip and extract the trace file http-wireshark-trace4-1.

- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.
- *Note:* If you are unable to run Wireshark on a live network connection, you can use the "classic" http-ethereal-trace-5 packet trace, or other additional traces, as notes in footnote 2, to answer the questions below.

Now let's examine the Wireshark output. You might want to first read up on HTTP authentication by reviewing the easy-to-read material on "HTTP Access Authentication Framework" at http://frontier.userland.com/stories/storyReader$2159

Answer the following questions[7]:

11. What is the server's response (status code and phrase) in response to the initial HTTP GET message from your browser? (1)

70038	1495.295080	172.16.4.135	128.119.245.12	HTTP	556 GET /wireshark-labs/protected_pages/HTTP-wireshark-file5.html HTTP/1.1
70049	1495.524900	128.119.245.12	172.16.4.135	HTTP	771 HTTP/1.1 401 Unauthorized (text/html)
70574	1508.890339	fe80::5edc:d429:191…	fe80::1909:7703:232…	HTTP/X…	807 POST /51003533-138a-4f58-a34a-0a8432d4f9c7/ HTTP/1.1

Status Code 401 Pharse Unauthorized

12. When your browser's sends the HTTP GET message for the second time, what new field is included in the HTTP GET message? (1)

```
Hypertext Transfer Protocol
  GET /wireshark-labs/protected_pages/HTTP-wireshark-file5.html HTTP/1.1\r\n
      Request Method: GET
      Request URI: /wireshark-labs/protected_pages/HTTP-wireshark-file5.html
      Request Version: HTTP/1.1
    Host: gaia.cs.umass.edu\r\n
    Connection: keep-alive\r\n
    Cache-Control: max-age=0\r\n
  Authorization: Basic d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcms=\r\n
      Credentials: wireshark-students:network
```

Authorization Field was added with GET request

The username (wireshark-students) and password (network) that you entered are encoded in the string of characters (d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcms=) following the "`Authorization: Basic`" header in the client's HTTP GET message. While it may appear that your username and password are encrypted, they are simply encoded in a format known as Base64 format. The username and password are *not* encrypted! To see this, go to http://www.motobit.com/util/base64-decoder-encoder.asp and enter the base64-encoded string d2lyZXNoYXJrLXN0dWRlbnRz and decode. *Voila!* You have translated from Base64 encoding to ASCII encoding, and thus should see your username! To view the password, enter the remainder of the string Om5ldHdvcms= and press

[7] If you're unable to run Wireshark on a live network connection, you can download the zip file http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip and extract the trace file http-wireshark-trace5-1.

decode. Since anyone can download a tool like Wireshark and sniff packets (not just their own) passing by their network adaptor, and anyone can translate from Base64 to ASCII (you just did it!), it should be clear to you that simple passwords on WWW sites are not secure unless additional measures are taken.

Fear not! As we will see in Chapter 8, there are ways to make WWW access more secure. However, we'll clearly need something that goes beyond the basic HTTP authentication framework!

NOTE: Please give valid reasons and a clear explanation in a few lines, for every question you answer!